

Leg-Entanglement Detection & Recovery

Project Overview

A plain-language summary for a non-technical reader

In one sentence: we built software that lets a four-legged robot notice — in real time, from its own body sensors — when one of its legs gets caught on a wire, net, or a person's hand, tell **which** leg and **how badly**, and then attempt to free it safely.

1. The problem

Four-legged robots increasingly work in messy, real-world spaces: rooms with cables, outdoor sites with netting or vegetation, or alongside people who might grab a leg. When a leg gets **caught**, the robot keeps trying to walk and fights the obstacle — and within a second or two it can stumble or fall. A fall means downtime, damage, and a safety risk.

Spotting this early is surprisingly hard:

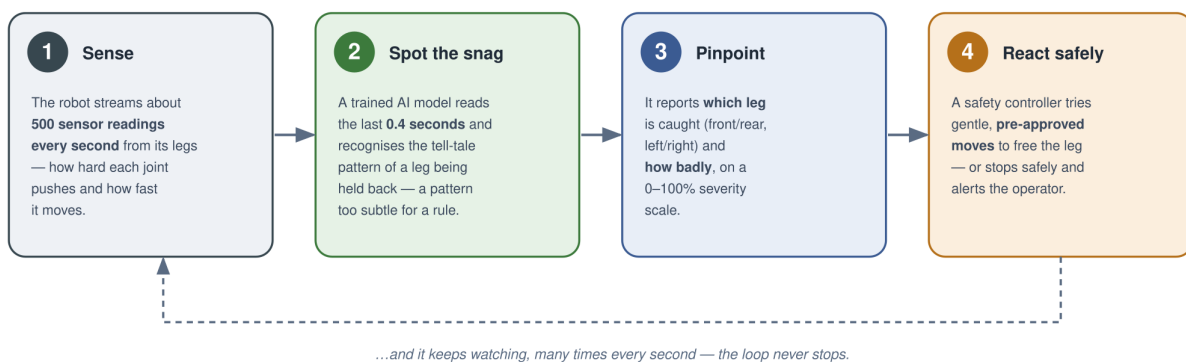
- **Cameras don't help much.** Thin wires and clear line are nearly invisible, and the robot's own body blocks the view of its legs.
- **A simple rule doesn't work.** The warning sign is subtle — a small, brief change in how hard a leg is straining — and it looks a lot like normal walking, standing, or the stiff pose the robot holds right after standing up. One fixed threshold either misses real snags or cries wolf constantly.

So instead of a camera or a fixed rule, we use the robot's **own sense of touch** — the hundreds of force and motion readings its legs already produce every second — and a small **trained model** that has learned what a real snag looks like.

2. Our solution

The system runs as a continuous loop on the robot, in four simple steps:

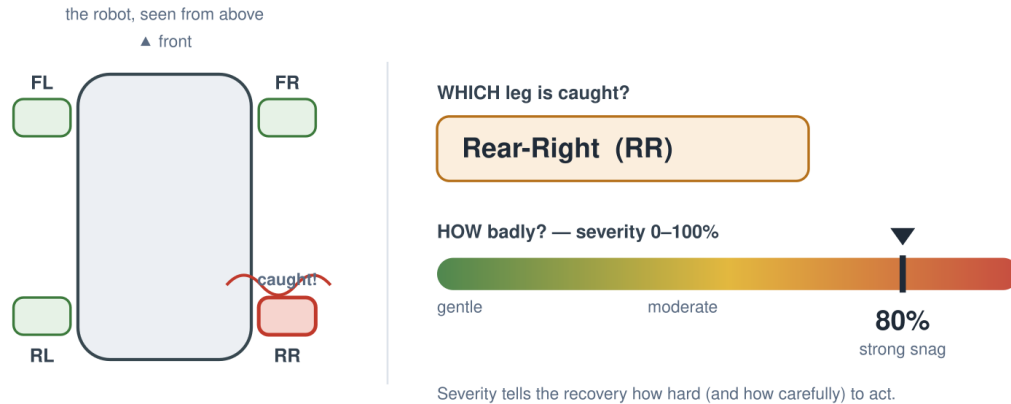
How it works — in four simple steps



What is the “AI model”? Think of it as a pattern-recogniser that we *taught* by showing it many real examples — recordings where a leg was caught, and recordings of normal walking, standing, and stopping. It learned the difference on its own, so it catches subtle cases a hand-written rule would miss. It only ever looks at the recent past, so it can run live on the robot.

The step that makes the system genuinely useful is **Step 3**: at every instant it reports *which* leg is caught and *how badly*.

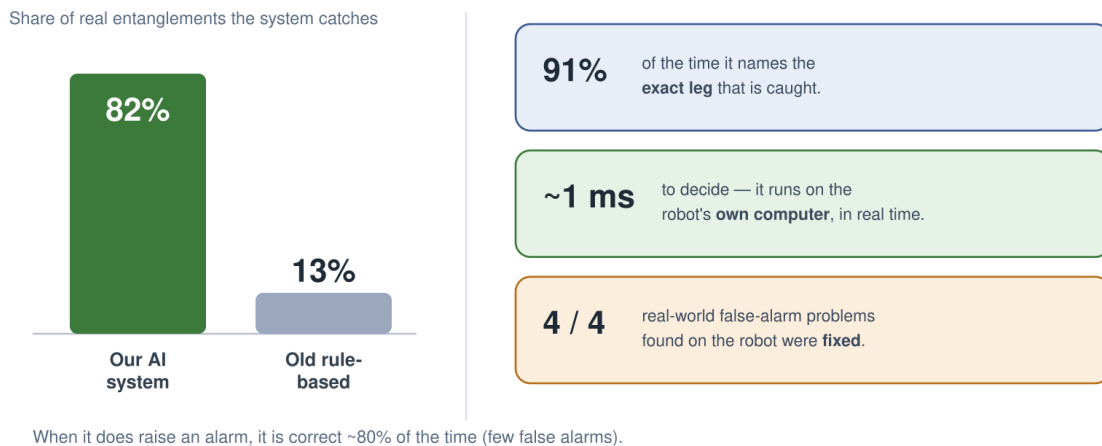
What the system reports at every instant



In **Step 4**, a separate **safety controller** uses that information to try gentle, pre-approved moves to free the leg — or stops the robot safely and alerts the operator. This recovery ships **switched off by default** and only logs what it *would* do, so it can be checked thoroughly before it is ever allowed to move a real robot. We deliberately kept the model **small and fast** so it runs on the robot's own modest computer in about a thousandth of a second, with no extra hardware and no network dependency.

3. Results — and what's new

Results at a glance



Measured carefully on held-out data the model had never seen during training:

- It **catches about 82%** of real entanglements — versus only **13%** for the older, rule-based approach on the same data.
- When it *does* raise an alarm, it is **correct about 80%** of the time.
- It names the **exact leg** correctly about **91%** of the time.
- It decides in about **1 millisecond** on the robot's own CPU — real-time.
- After the first on-robot trial we found **four** false-alarm situations (e.g. right after standing up, or

walking backwards); a little more data and re-training **eliminated all four**.

What makes this novel. Most prior work answers only “*is a leg caught?*”. Our system adds the two things a recovery controller actually needs:

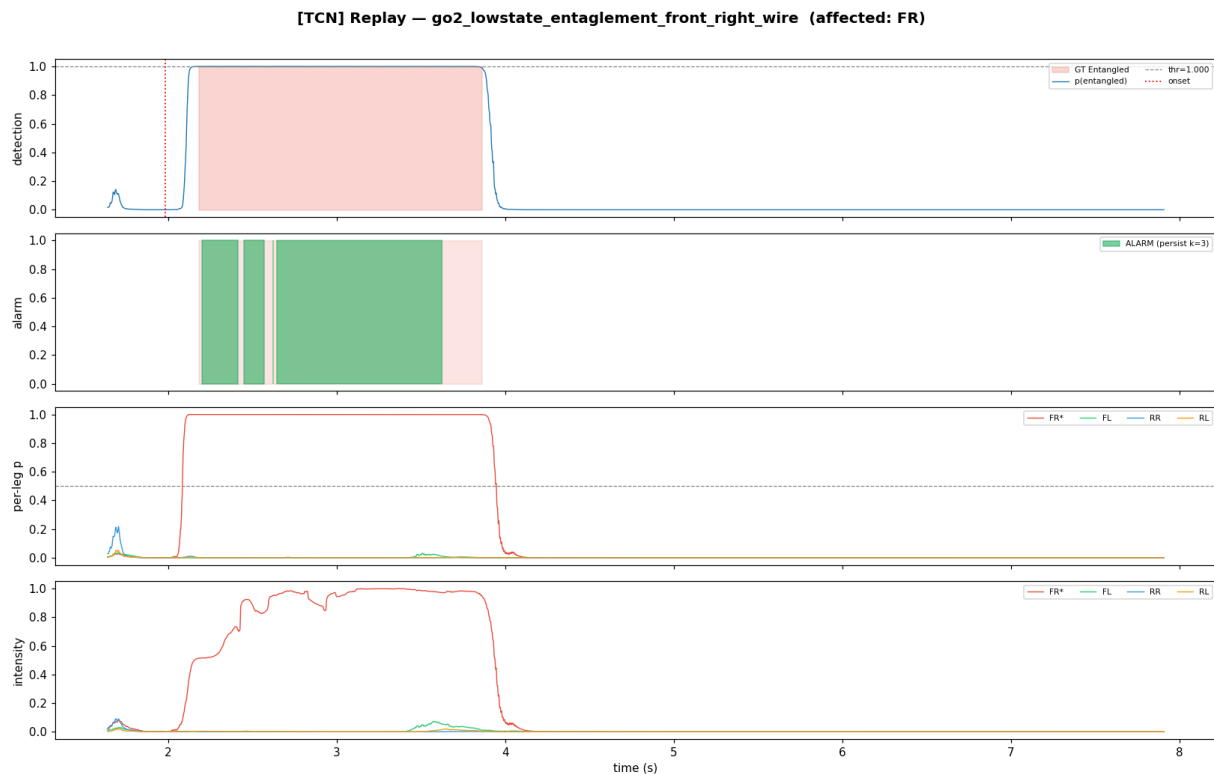
(a) The exact leg, not just “a leg”. The model points to one of the four specific legs, which tells the recovery *where* to act. We score this strictly — it must name exactly the right leg — and it succeeds $\sim 91\%$ of the time.

(b) A severity score (0–100%) — “**how badly**”. There are no ready-made severity measurements to learn from, so we compute severity from **physics**, blending two intuitive signals for the flagged leg:

1. **Strain-without-motion** — how hard the leg pushes while barely moving. A leg that pushes hard but goes nowhere is being held by *something*.
2. **How unusual the motion looks** — how far the leg’s behaviour has drifted from normal walking.

These combine into one score that sits near **0% in normal walking/standing** and **rises smoothly as a snag worsens**. The recovery uses it to act gently for a light pinch and more cautiously for a strong snag.

The system in action. Below is a real recording of a front-right leg caught on a wire. At the snag, all three outputs light up together — the alarm, the correct leg, and the severity — and fall back to zero the moment the leg is released; the other legs stay quiet.



Top to bottom: alarm confidence; the alarm; which leg (front-right, red); and severity — all rising at the snag and clearing at release.

4. Safety by design

- **Off by default.** Recovery motion is disabled unless an operator turns it on; otherwise it only *logs*

what it would do.

- **Only pre-approved moves** from the manufacturer's own documented commands, so the robot stays within its built-in balance limits.
- **Always a safe exit:** an emergency-stop makes the robot go limp instantly, and any fault or timeout falls back to that safe state automatically.
- **Detection is watch-only:** the detector never commands motion; it only reports.

5. Where we are, and what's next

Proven so far (measured): the detection accuracy, the exact-leg and severity outputs, the real-time speed on the robot's CPU, and the elimination of the four field false-alarm cases. The recovery decision-logic is fully tested in simulation.

Still to validate on the real robot (honest caveat): whether the recovery *moves* actually free a caught leg in practice — the directions and strengths of those moves are sensible engineering choices but have **not yet been proven on hardware**. That is why recovery ships switched off. The next step is a staged robot trial: detector only → recovery in log-only mode → carefully supervised live recovery.

Biggest opportunity: collect more examples of the under-represented cases (especially front-right snags) — the clearest way to push accuracy higher.